

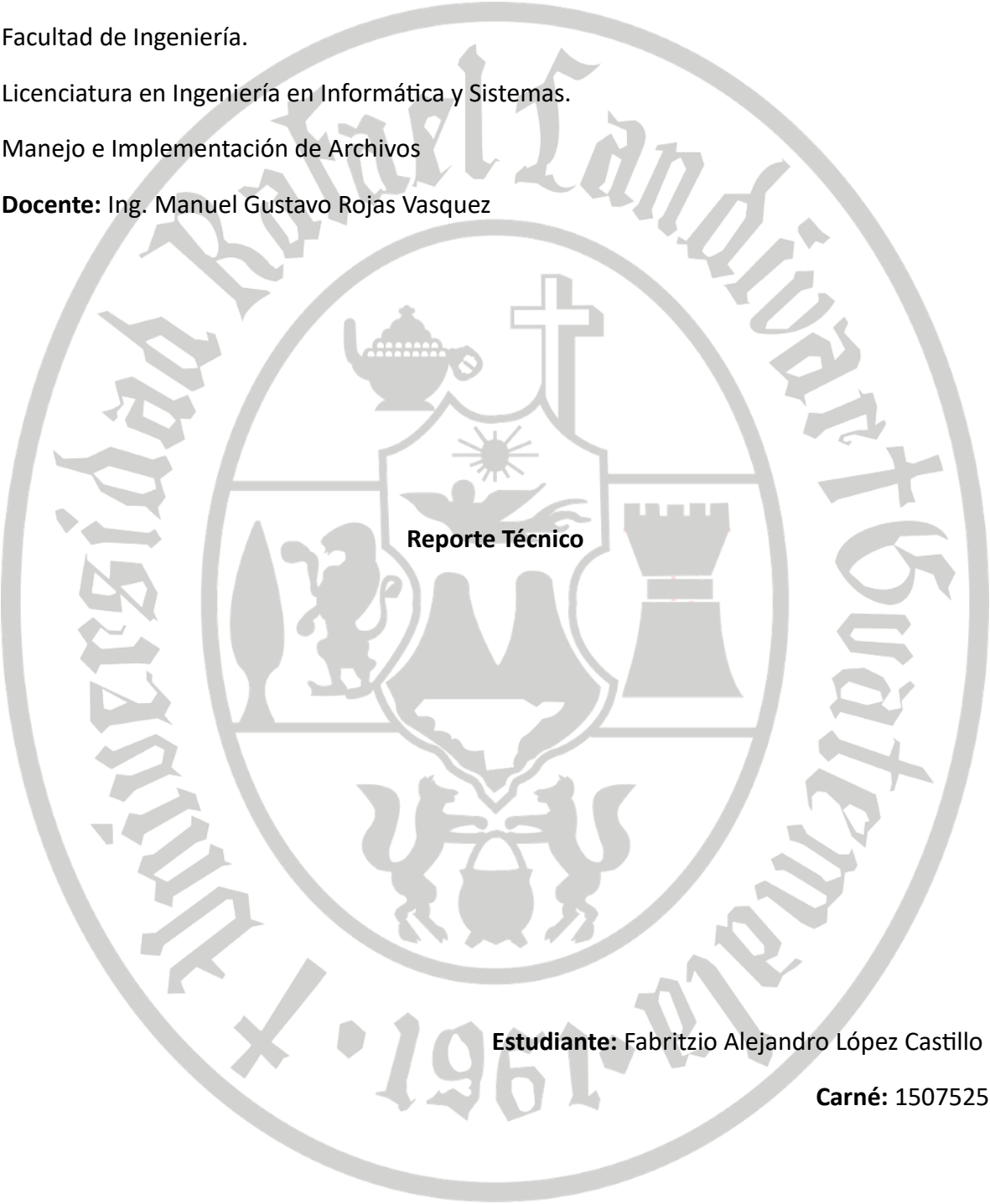
Universidad Rafael Landívar.

Facultad de Ingeniería.

Licenciatura en Ingeniería en Informática y Sistemas.

Manejo e Implementación de Archivos

Docente: Ing. Manuel Gustavo Rojas Vasquez

The seal of Universidad Rafael Landívar is a large, circular emblem in the background. It features a central shield with various symbols: a cross at the top, a sunburst, a mountain range, and a castle. The shield is flanked by two lions. The text "Universidad Rafael Landívar" is written in a circular path around the shield, and "1961" is at the bottom.

Reporte Técnico

Estudiante: Fabritzio Alejandro López Castillo

Carné: 1507525

Guatemala, 16 de septiembre de 2026

FORMATO ELEGIDO

Para el desarrollo de este proyecto se seleccionó JSON como formato definitivo para almacenar la configuración de usuario, descartando el formato CSV. La elección se basó en los siguientes criterios técnicos:

- **Facilidad de tipado de datos:** JSON guarda una relación directa (clave-valor) que se mapea de forma nativa a los diccionarios de Python. Respeta los tipos de datos: el tamaño fuente se guarda y recupera automáticamente como un entero, mientras que en CSV todo se leería como texto plano, requiriendo conversiones manuales propensas a errores al cargar la interfaz.
- **Robustez ante corrupción:** Al almacenar rutas de archivos (como la de la foto de perfil) o cadenas de texto complejas JSON encapsula automáticamente las comillas y caracteres especiales. En CSV una coma accidental en el nombre de un archivo corrompería la separación de las columnas y provocaría un error crítico en toda la lectura.
- **Legibilidad y Mantenimiento:** La estructura jerárquica de JSON facilita la lectura y la depuración del archivo en comparación con una fila larga de valores separados por comas que puede ser difícil de comprender sin sus encabezados ordenados según el tamaño de los datos.

IMPLEMENTACIÓN DE ESCRITURA SEGURA Y RESPALDO

Para evitar la corrupción del archivo en caso de un cierre inesperado del programa (como un corte de energía) se implementó un algoritmo de escritura segura de tres pasos utilizando las librerías json y os.

1. **Escritura Temporal:** Se escriben los datos en un archivo "config.tmp".
2. **Respaldo (.bak):** Si ya existe un config.json anterior, se renombra a "config.bak" para conservar una copia de seguridad.
3. **Consolidación:** Se renombra el archivo temporal config.tmp a config.json

Fragmento de código:

```
def guardar_configuracion(self, nuevos_datos):
    try:
        with open(self.archivo_temporal, "w", encoding="utf-8") as archivo:
            json.dump(nuevos_datos,
                      archivo,
                      indent=4,
                      ensure_ascii=False)

        if os.path.exists(self.archivo_final):
            os.replace(self.archivo_final, self.archivo_respaldo)

        os.replace(self.archivo_temporal, self.archivo_final)
        print("Configuración guardada de manera segura")
        return True
```

MANEJO DE ERRORES

El sistema está protegido mediante bloques try-except para evitar que excepciones de sistema operativo o formato causen un cierre inesperado. En los tres escenarios exigidos, la aplicación degrada a un comportamiento definido cargando un diccionario de valores por defecto.

- **Archivo ausente (FileNotFoundError):** Si el archivo no existe (primera ejecución) se captura el error y se cargan los valores estándar sin interrumpir el flujo.

```
Aviso: Archivo de configuración ausente. Cargando valores por defecto.
```

- **Archivo corrupto (json.JSONDecodeError):** Si el archivo existe pero su estructura JSON fue alterada incorrectamente de forma manual la librería json lanza un error de decodificación que es capturado e ignorado de forma segura.

```
Aviso: Archivo de configuración corrupto o con formato inválido. Cargando valores por defecto.
```

- **Sin permisos (PermissionError):** Si el archivo es de "solo lectura" o el sistema operativo niega el acceso, el programa captura el bloqueo.

```
Error: Falta de permisos de lectura. Cargando valores por defecto.
```

CODIFICACION UTF-8

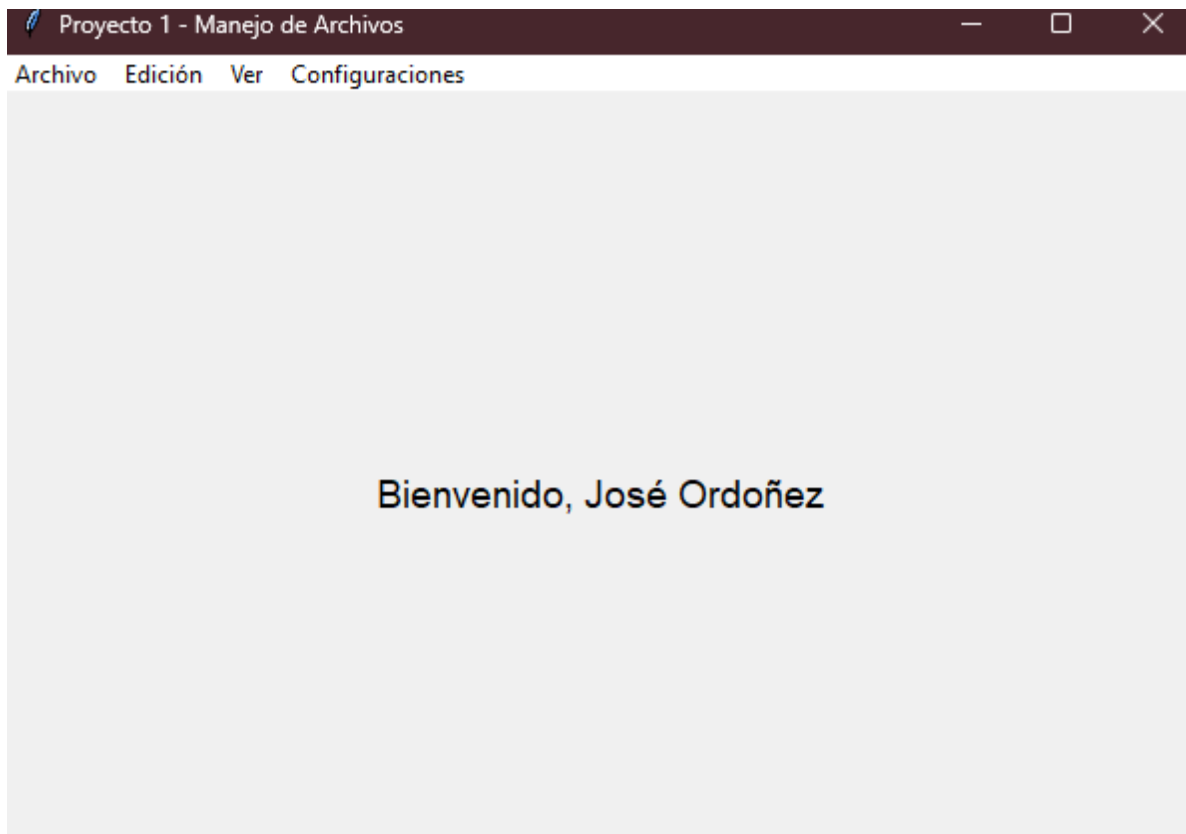
Para garantizar que los caracteres especiales no se pierdan o corrompan todas las operaciones de apertura de archivos incluyen el parámetro `encoding= utf-8` y en la escritura JSON se utiliza `ensure_ascii=False`.

A continuación, se muestra el contenido real de un archivo `config.json` generado por la aplicación, evidenciando el correcto manejo de tildes y caracteres del español en el nombre de usuario:

```
1 config.json > ...  
1 {  
2   "nombre_usuario": "José Ordoñez",  
3   "tema": "oscuro",  
4   "idioma": "en/en-US",  
5   "tamano_fuente": 10,  
6   "color_menu": "#ff0000",  
7   "color_letra": "#000000",  
8   "ruta_foto": "C:/Users/fabri/OneDrive/Imágenes/Saved Pictures/44-448127_universe-wallpaper-4k.jpg"  
9 }
```

APLICACIÓN

Ventana Principal con menús simulados y configuración:



Ventana funcional de configuraciones:

